53

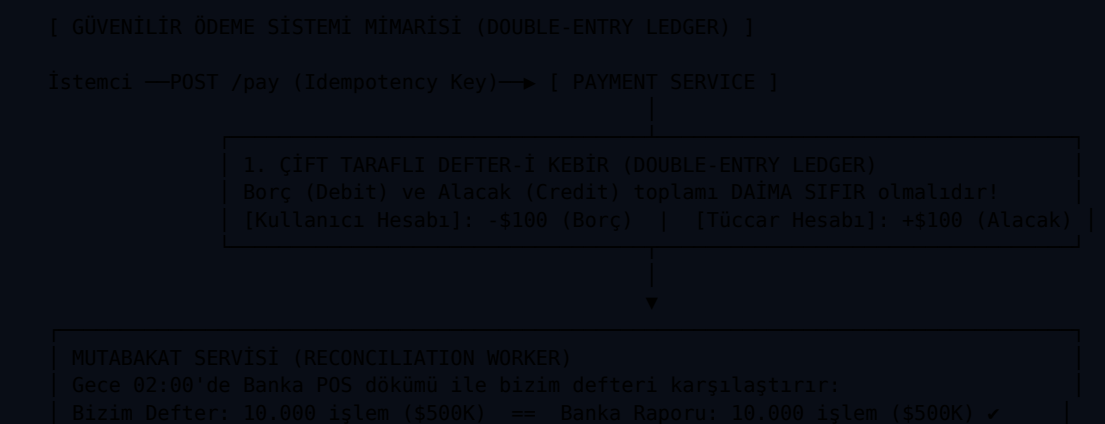
BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Ödeme Sistemi (Payment Gateway / Stripe): Çift Taraflı Muhasebe ve Mutabakat

Designing a Payment System (Stripe): Double-Entry Bookkeeping & Reconciliation

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Cüzdanınızdan 100 TL çıkarıp bir market sahibine verdiğinizde ne olur? Para havadan yok olmaz veya gökten zembille inmez! Sizin cebinizden 100 TL eksilir (-100 TL), marketçinin kasasına 100 TL girer (+100 TL). İkisini toplarsanız sonuç SIFIRDIR! 500 yıldır Venedikli tüccarların kullandığı 'Çift Taraflı Muhasebe' (Double-Entry Bookkeeping) budur. Stripe gibi devasa bir ödeme sisteminde tek bir 'bakiye' sayısı tutulmaz; tüm para hareketleri borç ve alacak olarak kaydedilir. Gece olunca da bankanın hesap dökümü ile bizim kayıtlarımız kuruşu kuruşuna karşılaştırılır (Mutabakat - Reconciliation)!

🇬🇧 ENGLISH (ELI5)

When you hand a $100 bill to a grocery clerk, cash doesn't materialize out of thin air or vanish into the void! Your wallet decreases by -$100 (Debit), while the register increases by +$100 (Credit). The net sum of the universe remains exactly ZERO! This is 500-year-old Double-Entry Bookkeeping. In a world-class payment platform like Stripe, you never store a simple mutable `balance = 500` column. You record immutable debit and credit ledger entries. At night, an automated auditor (Reconciliation Engine) cross-references internal ledgers against bank settlement files cent-for-cent!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Ödeme Sistemi Mimarisi Gereksinimleri: 1) Idempotency Garantisi: Ağ kopmalarında mükerrer para çekilmesini önlemek için istemci tarafından gönderilen `Idempotency-Key` zorunludur. 2) Çift Taraflı Muhasebe (Double-Entry Bookkeeping): Asla `UPDATE accounts SET balance = balance - 100` yapılmaz! Her işlem en az iki bacaklı bir defter kaydıdır (Ledger Entry): Biri Borç (Debit), diğeri Alacak (Credit). Sistem genelindeki tüm borç ve alacakların toplamı her an tam olarak sıfır (0) olmak zorundadır. 3) Mutabakat (Reconciliation): Bankalar ve PSP'ler (Stripe, Adyen, Visa) her gece CSV/XML formatında hesap mutabakat dosyaları (Settlement Files) gönderir. Arka plandaki mutabakat motoru, bizim yerel defterimizdeki işlemlerle bankanın bildirdiği işlemleri kuruşu kuruşuna eşleştirir; eksik veya hatalı işlemlerde alarm çalar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Payment Gateway Architecture: 1) Guaranteed Idempotency: Mitigates double-charging during network retries via client-supplied `Idempotency-Key` headers locked inside distributed Redis stores. 2) Double-Entry Bookkeeping Ledger: Strictly avoids mutable balance columns (`balance = balance - 100`). Money movements are immutably logged across dual ledger legs: Every transaction creates matching Debit and Credit entries across distinct accounts. Invariant: ∑ Debits - ∑ Credits = 0 across the global ledger at all times. 3) Reconciliation Engine: Financial partners (Visa, Chase, Adyen) transmit end-of-day settlement files. An automated asynchronous reconciliation pipeline cross-audits internal ledger transactions against partner clearing logs, flagging uncaptured funds or discrepancies for manual treasury review.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Çift Taraflı Muhasebe Defteri (Double-Entry Bookkeeping) Modeli
import java.math.BigDecimal;
import java.util.List;

public class DoubleEntryLedger {
    public enum EntryType { DEBIT, CREDIT }
    public record LedgerEntry(String accountId, EntryType type, BigDecimal amount) {}

    public static boolean validateTransaction(List<LedgerEntry> entries) {
        BigDecimal balance = BigDecimal.ZERO;
        for (LedgerEntry entry : entries) {
            if (entry.type() == EntryType.DEBIT) {
                balance = balance.subtract(entry.amount());
            } else {
                balance = balance.add(entry.amount());
            }
        }
        // Borç ve alacak toplamı SIFIR olmalıdır!
        return balance.compareTo(BigDecimal.ZERO) == 0;
    }
}

void main() {
    var amount = new java.math.BigDecimal("150.00");
    var entries = List.of(
        new DoubleEntryLedger.LedgerEntry("ACC_USER_ALİ", DoubleEntryLedger.EntryType.DEBIT, amount),
        new DoubleEntryLedger.LedgerEntry("ACC_MERCHANT_XYZ", DoubleEntryLedger.EntryType.CREDIT,
amount)
    );

    boolean balanced = DoubleEntryLedger.validateTransaction(entries);
    IO.println("Muhasebe Kaydı Dengeli mi? " + balanced + " (Borç = Alacak)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta para işlemlerini saklarken `FLOAT` veya `DOUBLE` veri tipi kullanacağınızı ASLA söylemeyin! Kayan noktalı sayılar (Floating-point) ikili sistemde yuvarlama hataları yaratır (0.1 + 0.2 = 0.30000000000000004). Milyonlarca dolarlık sistemlerde açık çıkar! Para daima `BIGINT` (kuruş/cent cinsinden) veya Java `BigDecimal` ile tutulmalıdır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never use FLOAT or DOUBLE primitives for currency in software engineering! Binary floating-point arithmetic introduces rounding inaccuracies that leak money. Always store currency as 64-bit BIGINT integers representing cents or Java `BigDecimal` with explicit rounding modes.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Idempotency Key ödeme sistemlerinde mükerrer para çekimini engelleyen ilk savunma hattıdır.
- Çift Taraflı Muhasebe (Double-Entry) paranın asla yoktan var veya vardan yok olmamasını garanti eder.
- Mutabakat (Reconciliation) motoru iç defter ile banka dökümlerini karşılaştırarak tutarlılık sağlar.

🎯 Key Takeaways for the Interview

- Idempotency keys prevent double-charging users during client network retries.
- Double-entry bookkeeping ensures every debit is mirrored by an exact credit.
- Reconciliation pipelines audit internal transactional state against external bank settlements.